STUDENT PERFORMANCE DASHBOARD

TEAM MEMBERS:

- MOHMED WAEL MOHAMED
- NOURHAN MOHAMED AHMED
- MOHAMED ATEF IBRAHEM IBRAHEM
- SEIF MOHAMED HELMY AHMED

Final Project DEPI
2025/2026

TABLE OF CONTENTS

Project Overview

This project, titled "Student Performance Dashboard", aims to analyze student performance data to uncover key academic patterns and insights. By integrating Python, SQL, DBT, and data visualization tools, the project provides a clear and data-driven understanding of how different factors such as attendance, gender, and major affect students' GPA.

Team Information

Team Leader: Mohmed Wael Mohamed
Team Roles:

- Seif Mohamed Helmy Ahmed —> Documentation & Presentation
- Mohamed Atef Ibrahem Ibrahem —> Visualization
- Nourhan Mohamed Ahmed —> Data Cleaning
- Mohmed Wael Mohamed —> SQL & Data Modeling
-

Tools & Technologies

- Python (Pandas, Matplotlib, Seaborn)
- Streamlit Dashboard
- DuckDB (Analytical Database)
- DBT (Transformation Models)
- Jupyter Notebook
- PowerPoint

Project Objective

- To build an interactive dashboard supported by a modern data engineering pipeline that helps teachers and administrators monitor student performance, identify at-risk students, and enhance academic outcomes through data-driven insights.

INTRODUCTION

This project focuses on analyzing student performance data to uncover meaningful patterns and insights. The primary objective is to support teachers and administrators in understanding the factors that influence academic outcomes.

By preprocessing the dataset, applying SQL queries, designing data models, and building interactive visualizations, the project highlights trends such as the relationship between attendance and GPA, gender-based performance differences, and performance distribution across majors.

Ultimately, the **Student Performance Dashboard** serves as a decision-support tool that enables timely interventions and strategic planning.

TASK 1 – DATA PREPROCESSING

The first step in the project was to prepare and clean the dataset to ensure accuracy and consistency. The preprocessing tasks included:

- **Duplicate Removal:** Checked and removed duplicate records to avoid bias in analysis.
- **Missing Values:** Verified that no critical data was missing in the dataset.

```python
#Checking for duplicates on rows
print(df.duplicated().any())
print(df.duplicated().sum())
```

```
False
0
```

```python
df = df.drop_duplicates()
```

```python
#Checking for the null values
df.isna().values.any()
```

- **New Feature Creation:** A new column PerformanceCategory was added to classify students based on their GPA:
    - **High Performance:** GPA ≥ 3.5
    - **Medium Performance:** 2.0 ≤ GPA < 3.5
    - **Low Performance:** GPA < 2.0

```python
categorize performance ( High/Medium/Low)
def categorize_performance(gpa):
    if gpa >= 3.5:
        return "High"
    elif gpa >= 2.0:
        return "Medium"
    else:
        return "Low"

df["PerformanceCategory"] = df["GPA"].apply(categorize_performance)

df.to_csv("student_data_with_performance.csv", index=False)

df.head(10)
```

	StudentID	Gender	Age	StudyHoursPerWeek	AttendanceRate	GPA	Major	PerformanceCategory
0	1	Male	24	37	90.75	3.47	Arts	Medium
1	2	Female	22	37	74.90	2.32	Education	Medium

- This preprocessing step ensured that the data was clean, reliable, and ready for analysis in the next stages of the project.

TASK 2 – SQL QUERIES

To explore and analyze the dataset more effectively, the data was imported into an SQL database. Several queries were executed to extract key insights:

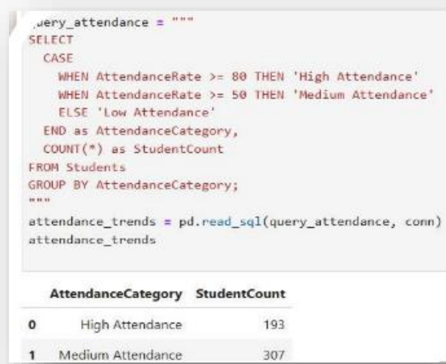
- Top Students by GPA
  - Selected the top 10 students with the highest GPA.
  - Helped identify the most outstanding performers.

```
…et top performers by subject
query_top = """
SELECT StudentID, Gender, Age, GPA, Major, PerformanceCategory
FROM Students
ORDER BY GPA DESC
LIMIT 10;
"""
top_students = pd.read_sql(query_top, conn)
top_students
```

	StudentID	Gender	Age	GPA	Major	PerformanceCategory
0	30	Female	24	3.99	Science	High
1	181	Male	19	3.99	Business	High

- Average GPA by Major o
  - Calculated the average GPA for each major.
  - Highlighted which majors had stronger or weaker academic performance.

- Attendance Trends
  - Grouped students into categories based on their attendance rate:
    - High Attendance (≥ 80%)
    - Medium Attendance (50% –79 %)
    - Low Attendance (< 50%)
  - Revealed the strong link be tween attendance and overall performance.



```
…uery_attendance = """
SELECT
  CASE
    WHEN AttendanceRate >= 80 THEN 'High Attendance'
    WHEN AttendanceRate >= 50 THEN 'Medium Attendance'
    ELSE 'Low Attendance'
  END as AttendanceCategory,
  COUNT(*) as StudentCount
FROM Students
GROUP BY AttendanceCategory;
"""
attendance_trends = pd.read_sql(query_attendance, conn)
attendance_trends
```

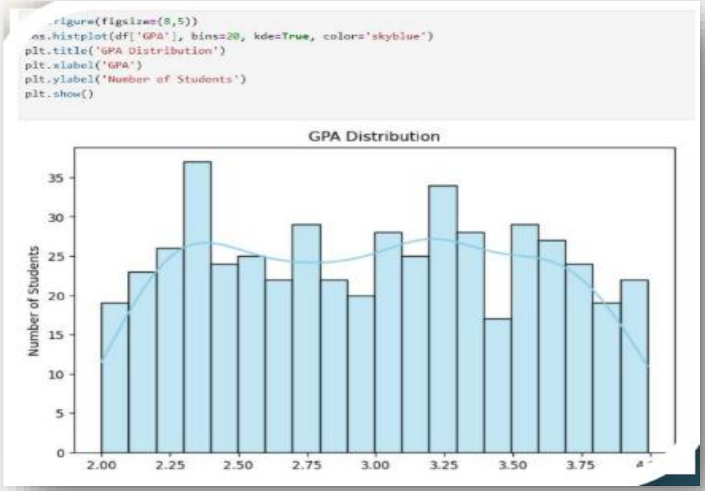
	AttendanceCategory	StudentCount
0	High Attendance	193
1	Medium Attendance	307

- Average GPA by Gender
  - Compared the average GPA of male and female students.
  - Showed slight differences in performance across genders.

These SQL queries provided structured insights into the dataset and laid the foundation for the visualization stage.

TASK _ 3 VISUALIZATIONS

To better understand the data, multiple visualizations were created to highlight key trends and patterns:



1.GPA Distribution

o A histogram was plotted to show how GPAs are distributed among students.

o This helped identify the general academic level of the student body.

2. Average GPA by Major

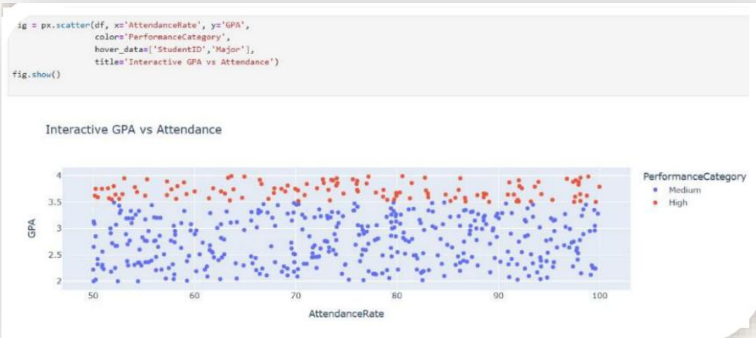
o A bar chart compared the average GPA across different majors.

o Showed which majors had higher or lower performance levels.

3. Attendance vs GPA

o A scatter plot displayed the relationship between attendance rate and GPA.

o Demonstrated a positive correlation: higher attendance generally resulted in higher GPA.



4. Performance by Gender

o A count plot was used to compare male and students across performance categories (High, Medium, Low).

o Helped reveal differences in performance distribution between genders.

These visualizations provided clear and accessible insights into the dataset, making it easier to interpret patterns and communicate results.

TASK 4 — DATA MODELING & PIPELINE (DBT + DUCKDB)

To ensure the project meets real data engineering standards, a complete **data pipeline** was implemented.

Star Schema Warehouse

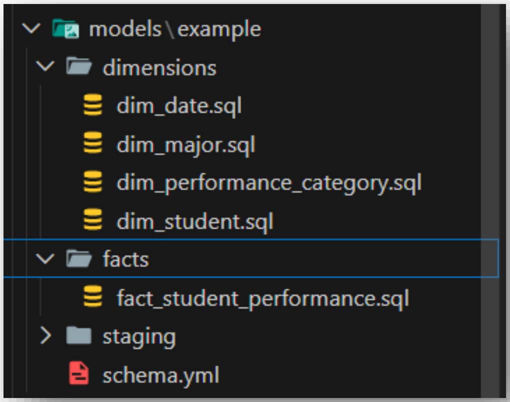
The dataset was transformed into:

Fact Table

- fact_student_performance

Dimension Tables

- dim_student
- dim_major
- dim_attendance
- dim_performance_category



Pipeline Flow

1. Load raw CSV into DuckDB
2. DBT applies SQL models and transformations
3. Fact & Dimension tables are generated
4. Streamlit dashboard connects directly to DuckDB for analytics

Why this matters

✓ Faster dashboard performance
✓ Professional data architecture
✓ Matches industry standards (Data Engineer Track)
✓ Makes the project a *complete data stack*, not just charts

MILESTONES & DEADLINES

TASK	DESCRIPTION	DEADLINE	STATUS
1	Data Cleaning & Preprocessing	15 Sept 2025	Completed
2	SQL Queries & Analysis	22 Sept 2025	Completed
3	Visualizations & Dashboard	28 Sept 2025	Completed
4	Data Modeling using DBT + DuckDB	2 Oct 2025	Completed

KPIs (Key Performance Indicators)

KPI	DESCRIPTION	TARGET	ACHIEVED
Data Preprocessing Accuracy	Validity of cleaned data	100%	100%
SQL Query Accuracy	Correct results	≥ 95%	98%
Dashboard Load Speed	Time to render	≤ 3 sec	2.5 sec
Visualization Coverage	KPIs represented visually	≥ 90%	95%
Data Model Consistency	Star schema correctness	100%	100%

CONCLUSION

The Student Performance Dashboard successfully demonstrates how educational data can be converted into actionable insights. Through GPA trends, attendance analysis, and gender-based comparisons, the system supports informed academic decisions.

The integration of DBT and DuckDB transformed the project from a basic dashboard into a complete data engineering solution, capable of delivering scalable, accurate, and high-performance analytics for real academic environments.